

SQL 100 Questions - Data Engineer Prep

100 SQL Interview Questions for Data Engineers — with Real-World Scenarios & Explanations

Sample Tables Used Throughout This Guide

- 📁 Category 1: SQL Basics & SELECT
- 📁 Category 2: Filtering, Sorting & CASE Logic
- 📁 Category 3: Aggregate Functions, GROUP BY & HAVING
- 📁 Category 4: Joins
- 📁 Category 5: Subqueries
- 📁 Category 6: Window Functions
- 📁 Category 7: CTEs & Recursive Queries
- 📁 Category 8: String & Date Functions
- 📁 Category 9: Data Engineering Specific Scenarios
- 📁 Category 10: Real-World Business Scenarios & Performance
- 📖 How to Practice This Effectively

100 SQL Interview Questions for Data Engineers — with Real-World Scenarios & Explanations

Organized into 10 categories × 10 questions. Practice each with sample tables in your head (e.g., `orders`, `customers`, `employees`) before checking the answer.

Sample Tables Used Throughout This Guide

```
-- employees(emp_id, name, dept_id, salary, manager_id, hire_date)
-- departments(dept_id, dept_name)
-- orders(order_id, customer_id, order_date, amount, status)
-- customers(customer_id, name, city, signup_date)
-- products(product_id, product_name, category, price)
-- order_items(order_id, product_id, quantity)
```

📁 Category 1: SQL Basics & SELECT

Q1. Scenario: You're given an `employees` table. Fetch all employees earning more than 50000.

```
SELECT * FROM employees WHERE salary > 50000;
```

Explanation: `WHERE` filters rows before aggregation. Basic filter on a single column.

Q2. Scenario: Get unique department IDs from the employees table.

```
SELECT DISTINCT dept_id FROM employees;
```

Explanation: `DISTINCT` removes duplicate values in the result set.

Q3. Scenario: List employee names in uppercase.

```
SELECT UPPER(name) AS emp_name FROM employees;
```

Explanation: String function applied at query time; doesn't change stored data.

Q4. Scenario: Fetch top 5 highest-paid employees.

```
SELECT * FROM employees ORDER BY salary DESC LIMIT 5;
```

Explanation: ORDER BY ... DESC + LIMIT gives top-N rows. (SQL Server uses TOP 5 instead.)

Q5. Scenario: Find employees whose name starts with 'A'.

```
SELECT * FROM employees WHERE name LIKE 'A%';
```

Explanation: % is a wildcard for zero-or-more characters; _ matches exactly one character.

Q6. Scenario: Get employees hired between Jan 2023 and Dec 2023.

```
SELECT * FROM employees
WHERE hire_date BETWEEN '2023-01-01' AND '2023-12-31';
```

Explanation: BETWEEN is inclusive of both bounds — a common gotcha in interviews.

Q7. Scenario: Find employees with no manager assigned (NULL manager_id).

```
SELECT * FROM employees WHERE manager_id IS NULL;
```

Explanation: NULL can never be compared with = ; always use IS NULL / IS NOT NULL .

Q8. Scenario: Rename output column salary as monthly_salary in result.

```
SELECT name, salary AS monthly_salary FROM employees;
```

Explanation: Aliasing improves readability of downstream reports/BI tools.

Q9. Scenario: Combine first and last name into a full name column (assume first_name , last_name).

```
SELECT CONCAT(first_name, ' ', last_name) AS full_name FROM
employees;
```

Explanation: CONCAT merges strings; in some DBs you can also use || .

Q10. Scenario: Get employees NOT in department 10 or 20.

```
SELECT * FROM employees WHERE dept_id NOT IN (10, 20);
```

Explanation: NOT IN excludes rows matching any value in the list — careful with NULLs in the list (can cause unexpected empty results).

📁 Category 2: Filtering, Sorting & CASE Logic

Q11. Scenario: Categorize employees as 'High'/'Medium'/'Low' earners.

```
SELECT name,
CASE
    WHEN salary > 80000 THEN 'High'
    WHEN salary BETWEEN 40000 AND 80000 THEN 'Medium'
    ELSE 'Low'
END AS salary_band
FROM employees;
```

Explanation: `CASE WHEN` is SQL's if-else — very common in DE for creating derived business categories.

Q12. Scenario: Sort orders by amount descending, then by order_date ascending for ties.

```
SELECT * FROM orders ORDER BY amount DESC, order_date ASC;
```

Explanation: Multiple sort keys resolve ties using the next column in sequence.

Q13. Scenario: Find orders placed in the last 30 days.

```
SELECT * FROM orders WHERE order_date >= CURRENT_DATE - INTERVAL
'30 days';
```

Explanation: Date arithmetic differs by engine (`DATEADD` in SQL Server, `INTERVAL` in Postgres/MySQL).

Q14. Scenario: Find customers whose city is either 'Hyderabad' or 'Chennai'.

```
SELECT * FROM customers WHERE city IN ('Hyderabad', 'Chennai');
```

Explanation: `IN` is shorthand for multiple `OR` conditions.

Q15. Scenario: Get orders where amount is NULL or zero.

```
SELECT * FROM orders WHERE amount IS NULL OR amount = 0;
```

Explanation: Data quality check — common first step in DE pipelines to flag bad records.

Q16. Scenario: Filter products priced between 100 and 500, excluding category 'Electronics'.

```
SELECT * FROM products
WHERE price BETWEEN 100 AND 500 AND category <> 'Electronics';
```

Explanation: Combining range filter with exclusion using `<>` (not equal).

Q17. Scenario: Find the 3rd highest salary without using LIMIT/OFFSET.

```

SELECT MIN(salary) FROM (
    SELECT DISTINCT salary FROM employees ORDER BY salary DESC LIMIT
3
) t;

```

Explanation: Common interview trick question — nested subquery finds Nth highest value.

Q18. Scenario: Get employees sorted by department, and within each department by salary descending.

```

SELECT * FROM employees ORDER BY dept_id, salary DESC;

```

Explanation: Sort order is hierarchical — left-to-right column priority.

Q19. Scenario: Retrieve rows where customer name has 'raj' anywhere (case-insensitive).

```

SELECT * FROM customers WHERE LOWER(name) LIKE '%raj%';

```

Explanation: Wrapping column in `LOWER()` ensures case-insensitive match across engines that are case-sensitive by default.

Q20. Scenario: Get all orders except those with status 'CANCELLED'.

```

SELECT * FROM orders WHERE status != 'CANCELLED';

```

Explanation: Simple exclusion filter — but remember NULL status rows will be silently dropped too; add `OR status IS NULL` if needed.

📁 Category 3: Aggregate Functions, GROUP BY & HAVING

Q21. Scenario: Find total revenue per customer.

```

SELECT customer_id, SUM(amount) AS total_revenue
FROM orders GROUP BY customer_id;

```

Explanation: `GROUP BY` collapses rows into groups; aggregate functions summarize each group.

Q22. Scenario: Find departments with more than 10 employees.

```

SELECT dept_id, COUNT(*) AS emp_count
FROM employees GROUP BY dept_id
HAVING COUNT(*) > 10;

```

Explanation: `WHERE` filters rows before grouping; `HAVING` filters groups after aggregation — a classic interview distinction.

Q23. Scenario: Find the average order value per month.

```

SELECT DATE_TRUNC('month', order_date) AS month, AVG(amount) AS
avg_order_value
FROM orders GROUP BY DATE_TRUNC('month', order_date);

```

Explanation: `DATE_TRUNC` buckets timestamps into month-level granularity for trend reporting.

Q24. Scenario: Find the number of distinct customers who ordered each product.

```
SELECT product_id, COUNT(DISTINCT customer_id) AS unique_buyers
FROM order_items oi
JOIN orders o ON oi.order_id = o.order_id
GROUP BY product_id;
```

Explanation: `COUNT(DISTINCT ...)` avoids double-counting a customer who bought the same product multiple times.

Q25. Scenario: Find max and min order amount per customer.

```
SELECT customer_id, MAX(amount) AS max_order, MIN(amount) AS
min_order
FROM orders GROUP BY customer_id;
```

Explanation: Multiple aggregates can be computed together in a single GROUP BY pass — efficient for reporting.

Q26. Scenario: Find departments where total salary exceeds 5,00,000.

```
SELECT dept_id, SUM(salary) AS total_salary
FROM employees GROUP BY dept_id
HAVING SUM(salary) > 500000;
```

Explanation: HAVING can reference aggregate expressions directly, unlike WHERE.

Q27. Scenario: Count orders by status.

```
SELECT status, COUNT(*) AS order_count
FROM orders GROUP BY status;
```

Explanation: Common dashboard query — status distribution for operational monitoring.

Q28. Scenario: Find customers who placed more than 5 orders in the last 90 days.

```
SELECT customer_id, COUNT(*) AS order_count
FROM orders
WHERE order_date >= CURRENT_DATE - INTERVAL '90 days'
GROUP BY customer_id
HAVING COUNT(*) > 5;
```

Explanation: Combines row-level filter (WHERE) with group-level filter (HAVING) — a very common real-world pattern.

Q29. Scenario: Get revenue contribution percentage per category.

```
SELECT category,
SUM(price * quantity) AS category_revenue,
SUM(price * quantity) * 100.0 / SUM(SUM(price * quantity)) OVER
() AS pct_of_total
FROM products p
JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY category;
```

Explanation: Combines `GROUP BY` with a window function (`OVER()`) to compute each group's share of the overall total.

Q30. Scenario: Find the second highest total revenue among all customers.

```
SELECT customer_id, total_revenue FROM (  
    SELECT customer_id, SUM(amount) AS total_revenue,  
           DENSE_RANK() OVER (ORDER BY SUM(amount) DESC) AS rnk  
    FROM orders GROUP BY customer_id  
    ) t WHERE rnk = 2;
```

Explanation: `DENSE_RANK` handles ties correctly (unlike plain `LIMIT/OFFSET`, which can skip valid tied values).

📁 Category 4: Joins

Q31. Scenario: List all orders with customer names.

```
SELECT o.order_id, c.name, o.amount  
FROM orders o  
INNER JOIN customers c ON o.customer_id = c.customer_id;
```

Explanation: `INNER JOIN` returns only matching rows in both tables.

Q32. Scenario: List all customers, including those who never placed an order.

```
SELECT c.customer_id, c.name, o.order_id  
FROM customers c  
LEFT JOIN orders o ON c.customer_id = o.customer_id;
```

Explanation: `LEFT JOIN` keeps all left-table rows; unmatched right-side columns return `NULL` — essential for “find missing data” scenarios.

Q33. Scenario: Find customers who have NEVER placed an order.

```
SELECT c.customer_id, c.name  
FROM customers c  
LEFT JOIN orders o ON c.customer_id = o.customer_id  
WHERE o.order_id IS NULL;
```

Explanation: Classic “anti-join” pattern — `LEFT JOIN` + filter for `NULL` on the right side.

Q34. Scenario: Combine employee list with department names, including departments with zero employees.

```
SELECT d.dept_name, e.name  
FROM departments d  
LEFT JOIN employees e ON d.dept_id = e.dept_id;
```

Explanation: Direction of `LEFT JOIN` matters — here `departments` is the “keep-all” side.

Q35. Scenario: Get all combinations needed for a report matrix (e.g., every product x every month).

```
SELECT p.product_id, m.month  
FROM products p  
CROSS JOIN (SELECT DISTINCT DATE_TRUNC('month', order_date) AS  
            month FROM orders) m;
```

Explanation: `CROSS JOIN` produces the Cartesian product — useful for generating complete report grids before joining actual data.

Q36. Scenario: Self-join to find each employee’s manager’s name.

```
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.emp_id;
```

Explanation: Self-join uses the same table twice with aliases — common for hierarchical/org-chart data.

Q37. Scenario: Find orders that reference a customer_id not present in the customers table (orphan/data-quality check).

```
SELECT o.*
FROM orders o
LEFT JOIN customers c ON o.customer_id = c.customer_id
WHERE c.customer_id IS NULL;
```

Explanation: A core Data Engineering check — catching referential-integrity breaks before loading into a Gold layer.

Q38. Scenario: Full outer join of employees and departments to find mismatches on both sides.

```
SELECT e.name, d.dept_name
FROM employees e
FULL OUTER JOIN departments d ON e.dept_id = d.dept_id;
```

Explanation: `FULL OUTER JOIN` keeps unmatched rows from both sides — good for full reconciliation checks. (MySQL lacks native support; emulate via `UNION` of `LEFT` + `RIGHT JOIN`.)

Q39. Scenario: Join order_items with products to calculate total order value.

```
SELECT oi.order_id, SUM(oi.quantity * p.price) AS order_total
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
GROUP BY oi.order_id;
```

Explanation: Join first to enrich data, then aggregate — a very common ETL transformation pattern.

Q40. Scenario: Join 3 tables — orders, customers, and order_items — to build a flat reporting table.

```
SELECT o.order_id, c.name, oi.product_id, oi.quantity
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
JOIN order_items oi ON o.order_id = oi.order_id;
```

Explanation: Multi-table joins are the backbone of building “Gold layer” denormalized/flattened tables for BI.

📁 Category 5: Subqueries

Q41. Scenario: Find employees earning more than the average salary.

```
SELECT * FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Explanation: Scalar subquery — returns a single value used for comparison.

Q42. Scenario: Find customers who placed at least one order above ₹5000.

```
SELECT * FROM customers
WHERE customer_id IN (SELECT customer_id FROM orders WHERE amount
> 5000);
```

Explanation: Subquery in `IN` clause — filters based on a related table's condition.

Q43. Scenario: Find products never ordered.

```
SELECT * FROM products
WHERE product_id NOT IN (SELECT product_id FROM order_items);
```

Explanation: ⚠ Careful: if `order_items.product_id` has any NULLs, `NOT IN` returns zero rows unexpectedly — safer to use `NOT EXISTS`.

Q44. Scenario: Same as Q43, but written safely using `NOT EXISTS`.

```
SELECT * FROM products p
WHERE NOT EXISTS (
    SELECT 1 FROM order_items oi WHERE oi.product_id = p.product_id
);
```

Explanation: `NOT EXISTS` is NULL-safe and often performs better than `NOT IN` on large datasets.

Q45. Scenario: Find each department's highest-paid employee using a correlated subquery.

```
SELECT * FROM employees e
WHERE salary = (
    SELECT MAX(salary) FROM employees e2 WHERE e2.dept_id =
e.dept_id
);
```

Explanation: Correlated subquery re-executes per outer row — powerful but can be slow on huge tables (window functions often preferred, see Q52).

Q46. Scenario: Find customers whose total order amount exceeds ₹1,00,000 (subquery in `FROM`).

```
SELECT * FROM (
    SELECT customer_id, SUM(amount) AS total_spent
    FROM orders GROUP BY customer_id
) t
WHERE total_spent > 100000;
```

Explanation: Derived table (subquery in `FROM` clause) — lets you filter on an aggregated result.

Q47. Scenario: Find employees who earn more than their own department's average salary.


```

SELECT * FROM employees e
WHERE salary > (
    SELECT AVG(salary) FROM employees e2 WHERE e2.dept_id =
e.dept_id
);

```

Explanation: Correlated subquery comparing each row against a group-level benchmark.

Q48. Scenario: Check if a customer exists before inserting (data pipeline dedup check).

```

SELECT 1 WHERE EXISTS (
    SELECT 1 FROM customers WHERE customer_id = 101
);

```

Explanation: `EXISTS` stops scanning as soon as one match is found — efficient existence check, common in incremental load logic.

Q49. Scenario: Get the latest order per customer using a subquery.

```

SELECT * FROM orders o
WHERE order_date = (
    SELECT MAX(order_date) FROM orders o2 WHERE o2.customer_id =
o.customer_id
);

```

Explanation: Correlated subquery pattern to fetch “latest record per group” — foundational for SCD Type 1 logic.

Q50. Scenario: Nested subquery: Find departments where the average salary is above the company-wide average.

```

SELECT dept_id, AVG(salary) AS dept_avg
FROM employees
GROUP BY dept_id
HAVING AVG(salary) > (SELECT AVG(salary) FROM employees);

```

Explanation: Combines GROUP BY/HAVING with a scalar subquery for company-wide benchmark comparison.

📁 Category 6: Window Functions

Q51. Scenario: Rank employees by salary within each department.

```

SELECT name, dept_id, salary,
    RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS
salary_rank
FROM employees;

```

Explanation: `PARTITION BY` resets ranking per group; `RANK()` skips numbers after ties (1,1,3).

Q52. Scenario: Find each department’s top earner (window function alternative to Q45).

```

SELECT * FROM (
    SELECT *, RANK() OVER (PARTITION BY dept_id ORDER BY salary
DESC) AS rnk
    FROM employees
) t WHERE rnk = 1;

```

Explanation: Window functions computed once over the dataset — generally faster than correlated subqueries at scale.

Q53. Scenario: Compute running total of daily revenue.

```

SELECT order_date, SUM(amount) AS daily_total,
    SUM(SUM(amount)) OVER (ORDER BY order_date) AS running_total
FROM orders
GROUP BY order_date;

```

Explanation: `SUM() OVER (ORDER BY ...)` without `PARTITION BY` creates a cumulative running total across the whole ordered set.

Q54. Scenario: Find the difference between each order's amount and the previous order's amount for a customer.

```

SELECT customer_id, order_date, amount,
    amount - LAG(amount) OVER (PARTITION BY customer_id ORDER BY
order_date) AS diff_from_prev
FROM orders;

```

Explanation: `LAG()` looks at the previous row's value within the partition — used for trend/delta analysis.

Q55. Scenario: Find the next order date for each customer.

```

SELECT customer_id, order_date,
    LEAD(order_date) OVER (PARTITION BY customer_id ORDER BY
order_date) AS next_order_date
FROM orders;

```

Explanation: `LEAD()` is the mirror of `LAG()` — peeks at the following row.

Q56. Scenario: Assign row numbers to deduplicate records (keep only the latest per key) — very common DE task.

```

SELECT * FROM (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY
order_date DESC) AS rn
    FROM orders
) t WHERE rn = 1;

```

Explanation: `ROW_NUMBER()` gives a unique sequential number per partition — the standard deduplication pattern in Bronze→Silver transformations.

Q57. Scenario: Calculate a 3-day moving average of order amounts.

```

SELECT order_date, amount,
    AVG(amount) OVER (ORDER BY order_date ROWS BETWEEN 2 PRECEDING
AND CURRENT ROW) AS moving_avg_3d
FROM orders;

```

Explanation: `ROWS BETWEEN ... PRECEDING AND CURRENT ROW` defines a sliding window frame — key for trend smoothing in analytics.

Q58. Scenario: Divide customers into 4 spending quartiles.

```
SELECT customer_id, SUM(amount) AS total_spent,  
       NTILE(4) OVER (ORDER BY SUM(amount) DESC) AS quartile  
FROM orders GROUP BY customer_id;
```

Explanation: `NTILE(n)` splits ordered rows into n roughly equal buckets — common for customer segmentation.

Q59. Scenario: Find the percentage contribution of each order to the customer's total spend.

```
SELECT customer_id, order_id, amount,  
       amount * 100.0 / SUM(amount) OVER (PARTITION BY customer_id) AS  
pct_of_customer_total  
FROM orders;
```

Explanation: Window aggregate in the denominator lets you compute per-row share of a group total without collapsing rows.

Q60. Scenario: Find the first order date and last order date per customer, alongside every row (without collapsing).

```
SELECT *,  
       FIRST_VALUE(order_date) OVER (PARTITION BY customer_id ORDER BY  
order_date) AS first_order,  
       LAST_VALUE(order_date) OVER (PARTITION BY customer_id ORDER BY  
order_date  
       ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS  
last_order  
FROM orders;
```

Explanation: `LAST_VALUE` needs an explicit full-frame (`UNBOUNDED FOLLOWING`) — a very common gotcha; without it, `LAST_VALUE` just returns the current row.

📁 Category 7: CTEs & Recursive Queries

Q61. Scenario: Use a CTE to simplify a multi-step revenue calculation.

```
WITH customer_revenue AS (  
  SELECT customer_id, SUM(amount) AS total_revenue  
  FROM orders GROUP BY customer_id  
)  
SELECT * FROM customer_revenue WHERE total_revenue > 50000;
```

Explanation: CTEs (`WITH`) improve readability vs. nested subqueries — the modern standard for multi-step SQL logic.

Q62. Scenario: Chain multiple CTEs together for a layered transformation.

```

WITH step1 AS (
    SELECT customer_id, SUM(amount) AS total_spent FROM orders GROUP
BY customer_id
),
step2 AS (
    SELECT customer_id, total_spent,
        CASE WHEN total_spent > 100000 THEN 'VIP' ELSE 'Regular' END
AS segment
    FROM step1
)
SELECT * FROM step2 WHERE segment = 'VIP';

```

Explanation: Multi-CTE chains mirror how Bronze→Silver→Gold transformation layers are structured, even within one query.

Q63. Scenario: Recursive CTE to build an employee-manager hierarchy.

```

WITH RECURSIVE org_chart AS (
    SELECT emp_id, name, manager_id, 1 AS level
    FROM employees WHERE manager_id IS NULL
    UNION ALL
    SELECT e.emp_id, e.name, e.manager_id, oc.level + 1
    FROM employees e
    JOIN org_chart oc ON e.manager_id = oc.emp_id
)
SELECT * FROM org_chart;

```

Explanation: Recursive CTEs traverse hierarchical/tree data (org charts, bill-of-materials, category trees) — the anchor member starts the recursion, the recursive member repeats until no new rows match.

Q64. Scenario: Use a CTE + window function to find each customer's rank by spend, then filter top 3 per city.

```

WITH ranked AS (
    SELECT c.city, c.customer_id, SUM(o.amount) AS total_spent,
        RANK() OVER (PARTITION BY c.city ORDER BY SUM(o.amount) DESC)
AS rnk
    FROM customers c JOIN orders o ON c.customer_id = o.customer_id
    GROUP BY c.city, c.customer_id
)
SELECT * FROM ranked WHERE rnk <= 3;

```

Explanation: Combining CTE + GROUP BY + window function — a very realistic “top-N per group” business report.

Q65. Scenario: Use a CTE to identify and remove duplicate rows (keep first occurrence).

```

WITH dedup AS (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY order_id ORDER BY
order_date) AS rn
    FROM orders
)
DELETE FROM orders WHERE order_id IN (
    SELECT order_id FROM dedup WHERE rn > 1
);

```

Explanation: ⚠ Syntax varies by engine (Postgres allows `DELETE ... USING`); the underlying dedup logic pattern is standard across DE pipelines.

Q66. Scenario: CTE to calculate month-over-month revenue growth %.

```
WITH monthly AS (  
    SELECT DATE_TRUNC('month', order_date) AS month, SUM(amount) AS  
revenue  
    FROM orders GROUP BY DATE_TRUNC('month', order_date)  
)  
SELECT month, revenue,  
    (revenue - LAG(revenue) OVER (ORDER BY month)) * 100.0 /  
LAG(revenue) OVER (ORDER BY month) AS growth_pct  
FROM monthly;
```

Explanation: Classic KPI calculation combining CTE (pre-aggregation) with `LAG()` for period-over-period comparison.

Q67. Scenario: Use a CTE to flag customers who haven't ordered in 60+ days (churn risk).

```
WITH last_order AS (  
    SELECT customer_id, MAX(order_date) AS last_order_date  
    FROM orders GROUP BY customer_id  
)  
SELECT * FROM last_order  
WHERE last_order_date < CURRENT_DATE - INTERVAL '60 days';
```

Explanation: A real churn-detection query pattern used in customer analytics pipelines.

Q68. Scenario: Recursive CTE to generate a date series for a report calendar (no gaps).

```
WITH RECURSIVE date_series AS (  
    SELECT DATE '2026-01-01' AS dt  
    UNION ALL  
    SELECT dt + INTERVAL '1 day' FROM date_series WHERE dt < DATE  
'2026-01-31'  
)  
SELECT * FROM date_series;
```

Explanation: Generating a full calendar table via recursion — useful to LEFT JOIN against so days with zero orders still show up as 0 (avoiding “gap” reporting errors).

Q69. Scenario: CTE to find products contributing to 80% of revenue (Pareto/ABC analysis).

```
WITH product_rev AS (  
    SELECT product_id, SUM(quantity * price) AS revenue  
    FROM order_items oi JOIN products p ON oi.product_id =  
p.product_id  
    GROUP BY product_id  
),  
ranked AS (  
    SELECT *, SUM(revenue) OVER (ORDER BY revenue DESC) * 1.0 /  
SUM(revenue) OVER () AS cumulative_pct  
    FROM product_rev  
)  
SELECT * FROM ranked WHERE cumulative_pct <= 0.8;
```

Explanation: Combines running-total window function inside a CTE to solve the classic “80/20 rule” business analysis.

Q70. Scenario: CTE-based incremental load pattern — find only new/changed rows since last run.

```
WITH source_data AS (  
    SELECT * FROM staging_orders WHERE updated_at > (SELECT  
        MAX(last_load_ts) FROM etl_control)  
    )  
    SELECT * FROM source_data;
```

Explanation: Core incremental-ETL pattern — compares source `updated_at` against a control table’s watermark instead of reloading the full table.

📁 Category 8: String & Date Functions

Q71. Scenario: Extract the year and month from an `order_date`.

```
SELECT order_id, EXTRACT(YEAR FROM order_date) AS yr,  
    EXTRACT(MONTH FROM order_date) AS mth  
FROM orders;
```

Explanation: `EXTRACT` pulls date-parts — standard ANSI SQL, widely supported.

Q72. Scenario: Calculate the number of days between order date and delivery date.

```
SELECT order_id, delivery_date - order_date AS days_to_deliver  
FROM orders;
```

Explanation: Date subtraction returns an interval/integer depending on engine (Postgres gives integer days directly).

Q73. Scenario: Format `order_date` as ‘DD-Mon-YYYY’ for a report.

```
SELECT TO_CHAR(order_date, 'DD-Mon-YYYY') AS formatted_date FROM  
orders;
```

Explanation: `TO_CHAR` (Postgres/Oracle) or `FORMAT / DATE_FORMAT` (SQL Server/MySQL) converts dates to display strings for reporting layers.

Q74. Scenario: Trim extra whitespace from customer names (common dirty-data issue).

```
SELECT TRIM(name) AS clean_name FROM customers;
```

Explanation: Data cleansing step — very common in Bronze→Silver transformations to standardize text fields.

Q75. Scenario: Replace all occurrences of ‘-’ with ‘/’ in a date-string column.

```
SELECT REPLACE(date_string, '-', '/') AS fixed_date FROM  
raw_import;
```

Explanation: `REPLACE` is a simple find-and-replace — often used before casting a messy string to a proper DATE type.

Q76. Scenario: Extract the domain from a customer's email address.

```
SELECT email, SUBSTRING(email FROM POSITION('@' IN email) + 1) AS
domain
FROM customers;
```

Explanation: Combining `POSITION` (find index of '@') with `SUBSTRING` — common pattern for parsing structured text fields.

Q77. Scenario: Pad order_id with leading zeros to make it 6 digits (e.g., 45 → 000045).

```
SELECT LPAD(order_id::TEXT, 6, '0') AS padded_order_id FROM
orders;
```

Explanation: `LPAD` is used for generating consistent ID formats — useful when exporting to external systems expecting fixed-width IDs.

Q78. Scenario: Find the length of each customer name (for validation — flag names > 50 chars).

```
SELECT name, LENGTH(name) AS name_length
FROM customers WHERE LENGTH(name) > 50;
```

Explanation: Data-quality validation — catching abnormally long/corrupted string values before loading downstream.

Q79. Scenario: Get the current timestamp for audit/logging columns in an ETL insert.

```
INSERT INTO audit_log (event, event_ts) VALUES ('Load Complete',
CURRENT_TIMESTAMP);
```

Explanation: `CURRENT_TIMESTAMP` (or `NOW()`) is standard for populating `created_at` / `loaded_at` audit columns in pipelines.

Q80. Scenario: Round order amount to 2 decimal places for currency reporting.

```
SELECT order_id, ROUND(amount, 2) AS amount_rounded FROM orders;
```

Explanation: `ROUND` avoids floating-point display issues in financial reports.

📁 Category 9: Data Engineering Specific Scenarios

Q81. Scenario: Implement UPSERT (insert if new, update if exists) — MERGE pattern used in Delta Lake/Fabric.

```
MERGE INTO target_customers t
USING staging_customers s
ON t.customer_id = s.customer_id
WHEN MATCHED THEN UPDATE SET t.name = s.name, t.city = s.city
WHEN NOT MATCHED THEN INSERT (customer_id, name, city) VALUES
(s.customer_id, s.name, s.city);
```

Explanation: This is THE most important pattern in modern Data Engineering (Delta Lake, Fabric, Databricks) — idempotent load logic that avoids duplicate inserts on re-runs.

Q82. Scenario: Implement SCD Type 2 (track historical changes) using MERGE.

```
MERGE INTO dim_customer t
USING staging_customer s
ON t.customer_id = s.customer_id AND t.is_current = TRUE
WHEN MATCHED AND (t.city <> s.city) THEN
    UPDATE SET t.is_current = FALSE, t.end_date = CURRENT_DATE
WHEN NOT MATCHED THEN
    INSERT (customer_id, city, start_date, end_date, is_current)
    VALUES (s.customer_id, s.city, CURRENT_DATE, NULL, TRUE);
```

Explanation: SCD Type 2 preserves history (old + new rows) instead of overwriting — critical for “as-of” reporting accuracy. A very common Fabric/DP-700 exam topic.

Q83. Scenario: Deduplicate records keeping the latest by `updated_at`, ready to write into a Silver Delta table.

```
SELECT * FROM (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY
updated_at DESC) AS rn
    FROM bronze_customers
) t WHERE rn = 1;
```

Explanation: The standard Bronze→Silver dedup transformation — same pattern as Q56, reused constantly in real pipelines.

Q84. Scenario: Validate schema/data quality — find rows where a required field is NULL before loading to Gold.

```
SELECT * FROM silver_orders
WHERE customer_id IS NULL OR order_date IS NULL OR amount IS NULL;
```

Explanation: Data Quality Framework check — corrupted rows get routed to a quarantine/error table instead of blocking the whole pipeline.

Q85. Scenario: Find late-arriving data — records with an `event_date` older than the `load_date` by more than 7 days.

```
SELECT * FROM staging_events
WHERE load_date - event_date > 7;
```

Explanation: Late-arriving data handling is a real-world streaming/batch challenge — flags records needing special reprocessing logic.

Q86. Scenario: Calculate a watermark value to use for the next incremental load.

```
SELECT MAX(updated_at) AS new_watermark FROM staging_orders;
```

Explanation: This value gets written to a control/metadata table and used as the filter in the next pipeline run (see Q70).

Q87. Scenario: Pivot order data — turn category rows into columns showing revenue per category per month.


```

SELECT month,
       SUM(CASE WHEN category = 'Electronics' THEN revenue ELSE 0 END)
AS electronics,
       SUM(CASE WHEN category = 'Clothing' THEN revenue ELSE 0 END) AS
clothing
FROM monthly_category_revenue
GROUP BY month;

```

Explanation: Manual PIVOT using conditional aggregation — portable across all SQL engines (some also offer native `PIVOT` syntax).

Q88. Scenario: Unpivot a wide table (columns per month) back into rows (long format) for easier analysis.

```

SELECT customer_id, 'Jan' AS month, jan_sales AS sales FROM
wide_table
UNION ALL
SELECT customer_id, 'Feb', feb_sales FROM wide_table;

```

Explanation: UNION ALL-based unpivot — turns a reporting-style wide table back into normalized long format for BI tools like Power BI.

Q89. Scenario: Find schema drift — columns present in staging but missing in the target table (metadata query, Postgres example).

```

SELECT column_name FROM information_schema.columns
WHERE table_name = 'staging_orders'
AND column_name NOT IN (
    SELECT column_name FROM information_schema.columns WHERE
table_name = 'target_orders'
);

```

Explanation: Metadata-driven pipelines query `information_schema` to auto-detect new columns before they break a rigid load — key concept for dynamic/metadata-driven ADF or Fabric pipelines.

Q90. Scenario: Partition pruning check — count rows loaded per partition date (validate a day's load).

```

SELECT partition_date, COUNT(*) AS row_count
FROM gold_sales
WHERE partition_date = CURRENT_DATE - 1
GROUP BY partition_date;

```

Explanation: Post-load validation step — confirms the expected row count landed for a given partition before marking the pipeline run successful.

📁 Category 10: Real-World Business Scenarios & Performance

Q91. Scenario: Find customers who bought Product A but NOT Product B (basket analysis).

```

SELECT DISTINCT o.customer_id
FROM orders o JOIN order_items oi ON o.order_id = oi.order_id
WHERE oi.product_id = 'A'
AND o.customer_id NOT IN (
    SELECT o2.customer_id FROM orders o2 JOIN order_items oi2 ON
o2.order_id = oi2.order_id
    WHERE oi2.product_id = 'B'
);

```

Explanation: Common marketing/cross-sell analysis — “bought X but not Y” targeting query.

Q92. Scenario: Find the top 3 selling products per category (real “Top-N per group” business report).

```

SELECT * FROM (
    SELECT category, product_name, SUM(quantity) AS total_qty,
        RANK() OVER (PARTITION BY category ORDER BY SUM(quantity)
DESC) AS rnk
    FROM order_items oi JOIN products p ON oi.product_id =
p.product_id
    GROUP BY category, product_name
) t WHERE rnk <= 3;

```

Explanation: One of the most frequently asked interview questions — window function + GROUP BY combo.

Q93. Scenario: Calculate customer retention rate month-over-month (cohort-style).

```

WITH first_purchase AS (
    SELECT customer_id, MIN(DATE_TRUNC('month', order_date)) AS
cohort_month
    FROM orders GROUP BY customer_id
)
SELECT fp.cohort_month, DATE_TRUNC('month', o.order_date) AS
activity_month,
    COUNT(DISTINCT o.customer_id) AS active_customers
FROM orders o JOIN first_purchase fp ON o.customer_id =
fp.customer_id
GROUP BY fp.cohort_month, DATE_TRUNC('month', o.order_date);

```

Explanation: Cohort analysis — a real business-analytics query pattern combining CTE + GROUP BY on two date dimensions.

Q94. Scenario: Explain why a query is slow and how to check its execution plan.

```

EXPLAIN ANALYZE
SELECT * FROM orders WHERE customer_id = 101;

```

Explanation: `EXPLAIN ANALYZE` shows the actual query plan (index scan vs. full table scan) — the starting point for any performance tuning conversation in interviews.

Q95. Scenario: Improve a slow filter query on a large orders table.

```

CREATE INDEX idx_orders_customer_id ON orders(customer_id);

```

Explanation: Indexing frequently filtered/joined columns dramatically reduces scan cost — a key performance-tuning topic (also relevant to Z-Ordering in Delta Lake).

Q96. Scenario: Avoid a common performance mistake — using a function on an indexed column in WHERE.

```
-- ✗ Slow (function on column disables index use):
SELECT * FROM orders WHERE YEAR(order_date) = 2026;

-- ✔ Fast (sargable — index-friendly):
SELECT * FROM orders WHERE order_date >= '2026-01-01' AND
order_date < '2027-01-01';
```

Explanation: Wrapping a column in a function makes a query “non-sargable,” preventing index usage — a favorite performance-tuning interview trick question.

Q97. Scenario: Find duplicate rows in a table based on multiple columns (before dedup).

```
SELECT customer_id, order_date, COUNT(*) AS dup_count
FROM orders
GROUP BY customer_id, order_date
HAVING COUNT(*) > 1;
```

Explanation: Standard duplicate-detection query — run before applying the ROW_NUMBER() dedup pattern (Q56, Q83).

Q98. Scenario: Calculate year-to-date (YTD) revenue as of each order date.

```
SELECT order_date, amount,
       SUM(amount) OVER (
         PARTITION BY EXTRACT(YEAR FROM order_date)
         ORDER BY order_date
       ) AS ytd_revenue
FROM orders;
```

Explanation: Combines PARTITION BY (reset per year) with a running SUM — common finance/reporting KPI.

Q99. Scenario: Find customers active in both Q1 and Q2 (intersection logic).

```
SELECT customer_id FROM orders WHERE order_date BETWEEN '2026-01-01' AND '2026-03-31'
INTERSECT
SELECT customer_id FROM orders WHERE order_date BETWEEN '2026-04-01' AND '2026-06-30';
```

Explanation: INTERSECT returns rows common to both result sets — clean way to express “active in both periods” without a self-join.

Q100. Scenario: Combine two differently-sourced order tables (e.g., online + in-store) into one unified report, removing duplicates.

```
SELECT order_id, customer_id, amount, 'Online' AS source FROM
online_orders
UNION
SELECT order_id, customer_id, amount, 'In-Store' AS source FROM
store_orders;
```

Explanation: UNION (not UNION ALL) removes exact duplicate rows across the combined sets — useful when consolidating multi-source data into one Gold table. Use UNION ALL instead when duplicates are

expected/acceptable and performance matters (no dedup overhead).

How to Practice This Effectively

1. Don't just read — **type every query yourself** in a free SQL sandbox (e.g., SQLite online, DB Fiddle, or your Fabric/DP-700 practice environment).
 2. For each question, **cover the answer first**, write your own version, then compare.
 3. Focus extra time on: **Window Functions (Cat. 6)** and **Data Engineering Scenarios (Cat. 9)** — these show up most in DP-700 and real DE interviews.
 4. Re-attempt the ones you got wrong after 2-3 days (spaced repetition).
-

Guide compiled — July 2026